

POWERSLIDER: Demand-Response-Aware LLM Serving via Dynamic SLOs and Joint Online Control

March 18, 2026

Abstract

As AI inference clusters grow to megawatt scale, grid operators increasingly require *demand response*: time-varying power caps and energy budgets that existing LLM serving systems cannot satisfy without unacceptable quality loss. We identify three challenges unique to demand-response-aware LLM serving: (1) static SLOs cannot simultaneously meet time-varying power constraints and preserve quality—*dynamic* SLOs that slide with the power budget are necessary; (2) the resulting design space (dynamic multi-SLO $\times$ stage-disaggregated DVFS $\times$ cache partitioning) is too large and too dynamic for the offline profiling used by DynamoLLM and TAPAS; (3) grid services impose both short-term power caps and long-term energy budgets, requiring dual time-horizon control. We present POWERSLIDER, which addresses all three challenges by jointly controlling GPU allocation, stage-aware DVFS, and KV-cache partitioning through a single online formulation. POWERSLIDER models cache hits with a concave power law, yielding closed-form water-filling allocations and stage-specific frequency targets via KKT conditions—eliminating offline profiling. A Lyapunov drift controller with separate power and energy virtual queues adapts decisions to time-varying carbon intensity, demand-response events, and workload shifts while respecting per-class SLO priorities. Using Azure serving traces and real grid carbon signals, POWERSLIDER reduces carbon emissions by **XX%** over carbon-agnostic baselines (and **XX%** over DynamoLLM/TAPAS under normal conditions), keeps **P95** latency within **5%**, and improves effective capacity by **XX $\times$** under grid service scenarios.

Keywords: LLM Inference, Carbon-Aware Computing, KV Cache, DVFS, Multi-SLO Scheduling

1 Introduction

The surging adoption of AI inference applications has drastically increased the need for high-performance data centers, which are predominantly powered by GPU clusters. The escalating energy use of these AI clusters is

putting a growing burden on existing power grids, especially during peak demand or when renewable energy generation is low. In the USA, for example, AI-related data center demand is expected to hit tens of gigawatts by 2030, worsening grid congestion and slowing down project rollouts. Currently, conservative power-system planning treats data centers as consistently high-power users, meaning this rising demand leads to large, costly grid infrastructure upgrades and long connection delays for new data centers.

Previous research on the demand response of computational load has predominantly been restricted to controlled academic settings. These studies primarily focused on CPU-based clusters running high-performance computing (HPC) applications [32], or used simulation and analytical models to demonstrate potential demand response capabilities [31, 26]. While providing valuable insights, this foundational work did not sufficiently address the distinct energy and power profiles and stringent, often rigid, performance demands characteristic of modern AI inference workloads running on GPUs or TPUs (often defined as SLO). Separately, industry efforts have shown that data centers can lower operational carbon emissions by curtailing compute resources for jobs with flexible performance requirements or by using traditional load shedding [17, 12, 22].

Incentive programs and contracts have been established between utility companies and AI data centers to facilitate participation in demand response and grid stabilization initiatives. However, the optimal methodology for maintaining user-level Service Level Objectives (SLOs) during these events remains an open research challenge. Our core contribution addresses demand response by introducing “Flex” user tiers in AI inference and leveraging scalable software and hardware controls for adaptive power modulation. Simple strategies, such as greedily scaling down GPU servers using Dynamic Voltage and Frequency Scaling (DVFS) or applying uniform power-capping across the fleet, are likely to disrupt workload performance and reduce workload goodput. Therefore, we propose the development of a novel software-hardware abstraction layer that communicates both the power cap signal and the workload SLO, and the design of new orchestration and

scheduling policies to minimize the impact of demand response on overall workload performance.

Unlike traditional web traffic or microservices requiring low-latency, LLM serving traffic can offer more flexibility, especially given the usage on batch tasks or semi-offline tasks like document analysis, synthetic data generation or compute-use research agents. Exploiting this flexibility is the central premise of our work. This trend is also emerging in the industry; for example, Anthropic has committed to covering 100% of grid upgrade costs tied to data centers, procuring new power generation to offset demand-driven price increases, and deploying curtailment systems to reduce strain during peak hours [2].

Serving these curtailment-enabled LLM workloads efficiently with the existing system and hardware stacks requires navigating three fundamental challenges that emerge specifically from the demand-response setting:

1. **Flex SLOs are necessary, not optional.** Grid services impose *time-varying* power caps (e.g., “reduce to 80% for 2 hours”). Under a *static* SLO assignment, the system either wastes 20–25% of capacity during unconstrained periods (if SLOs are always conservative) or cannot meet the power cap during DR events (if SLOs are set for normal operation). We show (§2.5.1, Figure ??) that no single static operating point can navigate both regimes—SLOs must *slide* with the power constraint. Industry trends already support tiered pricing: Anthropic offers “Fast” vs. “Standard” tiers at $6\times$ cost difference [3]; DeepSeek warns of slowdowns under load; and OpenAI routes free users to weaker models [?].
2. **Dynamic multi-SLO $\times$ demand response makes offline profiling infeasible.** DynamoLLM and TAPAS pre-profile a sparse set of (parallelism, frequency, batch-size) configurations offline. With dynamic SLOs (each class’s target moves independently), three-stage disaggregation, and continuous cache allocation, the joint configuration space grows to $O(10^9)$ —too large to profile, and the “right” point changes every 5 minutes as the power budget and load shift (§2.5.2, Figure 2).
3. **Short-term power caps and long-term energy budgets must be balanced jointly.** Grid programs enforce *both* an instantaneous power cap and a cumulative energy budget. Optimizing only for instantaneous power (DynamoLLM-style) wastes the energy budget and causes unnecessary degradation; optimizing only for energy violates the instantaneous cap during load peaks. A single formulation must respect both time horizons (§2.5.3, Figure ??).

These challenges interact: dynamic SLOs (Challenge 1) expand the design space that offline profiling cannot handle (Challenge 2), and the dual time-horizon constraint (Challenge 3) requires an online controller that

adapts SLO budgets in real time.

Existing systems address these dimensions in isolation. Carbon-aware or energy-aware schedulers defer batch workloads to low-carbon periods but ignore cache dynamics and assume static SLOs [18, 23]. DVFS controllers minimize power or energy via offline profiling or performance-modeling but lack multi-SLO awareness and cannot adapt to time-varying power budgets [29, 27, 19]. Multi-SLO serving frameworks like SGLang [33] or SLOs-Serve/PolyServe [5, 35] optimize for latency and throughput but do not consider energy or power objectives, and their static SLO across classes are not suitable for changing power demand response. KV cache optimizers maximize hit rates but are oblivious to energy and power constraints [11].

We present POWERSLIDER, a unified framework that addresses all three challenges by jointly optimizing frequency, allocation, and caching for demand-response-aware multi-SLO LLM serving. Our approach is grounded in three key technical observations:

We implement POWERSLIDER atop SGLang [19] and evaluate on Azure LLM inference traces combined with real grid carbon data from Electricity Maps. Our results demonstrate: YL *Numbers are for placeholder purpose*

- $XXX\%$ reduction in operational carbon emissions compared to carbon-agnostic baselines and 11–18% compared to DynamoLLM [19] and TAPAS [20];
- Less than 5% degradation in P95 and P99 latency for latency-sensitive classes; while introducing the Flex class can help save the total capacity by $XXX\%$, compared to no-flex class setting, especially under Grid Service or Grid Shortages.

2 Background

Table 1 compares the design knobs of POWERSLIDER with DynamoLLM [19] and Splitwise [14].

2.1 LLM Inference Architecture

Modern LLM inference proceeds in two distinct phases. The *prefill* phase processes the input prompt, computing attention over all input tokens in parallel. The *decode* phase generates output tokens autoregressively, with each token depending on all previous tokens. These phases have different computational characteristics: prefill is compute-bound with high arithmetic intensity, while decode is memory-bandwidth-bound due to loading large KV cache tensors.

Table 1: Comparison of design knobs across LLM inference systems. ✓: Supported, ○: Partial, ✗: Not supported.

Knob / Feature	DynamoLLM	Splitwise	SLOs-Serve	PolyServe	POWERSLIDER
<i>Existing inference knobs</i>					
DVFS (frequency scaling)	✓	✗	✗	✗	✓
Parallelism config (TP/PP)	✓	✓	✗	✗	✓
Chunk/Batch size tuning	✓	✗	✓	✗	✓
<i>New knobs for demand response</i>					
Prefill/[thinking]/decode disaggregation	✗	○	✗	○	✓
KV cache partitioning	✗	✗	✗	✗	✓
[Dynamic] Multi-SLO support	✗	✗	○	○	✓
[Energy-aware] rate-limit	✗	✗	✗	✗	✓
<i>Assumptions & design objective</i>					
Clairvoyant scheduler (known decode length)	✓	✓	✓	✓	✗
Constant SLO constraint	✓	✓	✓	✓	✗
Power/energy as optimization goal	✓	✓	✗	✗	✗
Time-varying power constraint	✗	✗	✗	✗	✓

2.2 GPU Frequency Scaling (DVFS)

Modern GPUs support dynamic voltage and frequency scaling (DVFS). Reducing frequency from $f_{\max}$ to f decreases power consumption but increases execution time. The relationship is workload-dependent: compute-bound kernels scale more linearly with frequency, while memory-bound kernels see diminishing returns.

For LLM inference, the prefill phase (compute-bound) benefits more from high frequency, while the decode phase (memory-bound) can often run at reduced frequency with minimal latency impact. This phase-dependent behavior creates opportunities for fine-grained frequency control.

2.3 Grid Services: The Primary Opportunity

Datacenters face a spectrum of power modulation scenarios [30], from millisecond-scale grid emergencies to week-long regional shortages. Figure ?? illustrates the taxonomy.

We focus on **grid services** (demand response): programs requiring enrolled customers to reduce load during peak periods, typically with *minutes-to-hours of advance notice* for 2–4-hour windows at 5–20% reduction magnitude. This scenario is the most relevant for LLM inference because: (i) the long lead time allows the serving system to *plan* rather than react, (ii) the moderate magnitude is achievable without shutting down entire clusters, and (iii) the growing importance of demand response—as grids incorporate more variable renewable generation—makes it a recurring operational constraint rather than a rare emergency [13]. Critically, grid services impose *both* an instantaneous power cap ($b_{\max}$, enforced at every in-

terval) and a cumulative energy budget ($E_{\max}$, enforced over the entire window), creating a dual time-horizon constraint that prior systems do not address (§2.5.3).

2.4 Workload Impact Modeling

A key challenge in power modulation is quantifying the impact on hosted workloads. We adopt the *Expected Shortfall* (ES) model from financial risk analysis [4], repurposed for cloud services.

For a service s with current capacity cap^* and post-action capacity cap' , the impact is the expected load that exceeds remaining capacity:

$$I_s(\text{cap}') = \frac{1}{\text{load}_{\text{avg}}} \int_{\text{cap}'}^{\text{cap}^*} \text{PDF}(\text{load}) \cdot (\text{load} - \text{cap}') d(\text{load}) \quad (1)$$

This metric captures the *risk* that load exceeds worker capacity, weighted by probability. Services with slack capacity (common due to peak provisioning) can absorb some reduction with zero impact, while right-sized services are more sensitive.

The total cost of a power modulation action across all services is:

$$\text{cost} = \sum_{s \in S} w_s \cdot I_s(\text{cap}') \quad (2)$$

where w_s reflects service priority based on properties such as user-facing vs. background, stateful vs. stateless, and resilience tier. This formulation guides POWERSLIDER’s allocation decisions to minimize workload disruption while meeting power targets.

2.5 Motivation: Three Challenges for Demand-Response-Aware LLM Serving

Operating LLM inference under grid service (demand response) constraints introduces three fundamental challenges that existing systems could not easily address. We illustrate each with a motivating example, then summarize how POWERSLIDER resolves them.

We illustrate each with a motivating example, then summarize how POWERSLIDER resolves them.

2.5.1 Challenge 1: Static SLOs Cannot Meet Time-Varying Power Constraints

Non-tiered and disaggregated baselines. Systems that treat all requests identically—whether monolithic (Non-tiered SLO) or disaggregated (SplitWise, no tiers)—degrade both classes uniformly. At 40% power reduction, both classes lose $\sim 44\%$ goodput (Figure 1a). Multi-SLO scheduling (PolyServe) improves the critical class modestly (by assigning less aggressive frequency reduction) but still drops flex-class goodput to 34%.

Priority scheduling: protection at the cost of starvation. The natural solution is to protect critical requests via strict priority (SplitWise + Priority). This succeeds for the critical class—75% goodput at 40% reduction—but *starves* flex requests, whose goodput collapses to just 14% at the same operating point (Figure 1b, $\Delta = 49$ pp gap with Flex SLO). The root cause is that priority scheduling has no mechanism to *exploit* the relaxed latency budget of the flex tier; it simply defers without being aware of the power implications and fluctuating constraints.

Flex SLO resolves the dilemma. POWERSLIDER introduces *elastic* SLO tiers: during a power-cap event, flex-class requests accept controlled latency inflation (up to $2\times$) rather than being dropped or starved. The result is that *both* classes sustain high goodput simultaneously—at 40% reduction the critical class retains 80% and the flex class retains 63%, a 49-percentage-point improvement over priority scheduling for the flex tier with only a 5 pp trade-off on the critical side. Industry practice already moves in this direction: Anthropic distinguishes “Fast” from “Standard” API tiers at a $6\times$ price difference [3]; DeepSeek warns of slowdowns under high traffic; and OpenAI routes free-tier users to weaker models under load. POWERSLIDER formalizes this with per-class impact bounds δ_i that *slide* with the power budget (§3.3).

2.5.2 Challenge 2: Offline Profiling Cannot Track the Expanding Design Space

DynamoLLM address power management by *offline profiling*: they enumerate combinations of parallelism degree, GPU frequency, and batch size, measure latency and power for each, and at runtime select the pre-profiled point that best fits the current power target. This strategy assumes a *fixed SLO* and a *small, static configuration space*.

Both assumptions break under demand response. First, with *dynamic* multi-SLO (Challenge 1), the target region on the power-latency Pareto front shifts as the power budget changes—independently for each SLO class (Figure 2). The pre-profiled grid is too sparse to interpolate accurately. Second, introducing cache partitioning and three-stage disaggregation (prefill/think/answer) expands the per-class decision space from 2D (parallelism $\times$ frequency) to 5D ($k_i \times f_i \times M_i \times \text{stage} \times \text{class}$). With 3 SLO classes, 3 stages, ~ 10 GPU-count options, and ~ 80 frequency steps per stage-class, the discrete joint space alone is $(|K| \times |F|)^{|\mathcal{P}| \times |\mathcal{C}|} \approx (10 \times 80)^9 \approx 10^{26}$ —orders of magnitude beyond DynamoLLM’s $\sim 10^3$ single-pipeline configurations and far too large to profile exhaustively—and the optimal point shifts every 5 minutes as the power budget and load change.

POWERSLIDER avoids offline profiling entirely by deriving *analytical* KKT conditions (§3.3.2) that yield closed-form cache water-filling and stage-specific frequency targets. The online solver evaluates these in < 100 ms per epoch, tracking the Pareto front continuously.

2.5.3 Challenge 3: Stage disaggregation

Reasoning-capable LLMs (e.g., DeepSeek-R1) generate a long *thinking* chain before emitting the user-visible answer. From a serving perspective, thinking tokens are autoregressive like decode tokens but far more memory-intensive—they involve extended KV-cache growth. This creates an architectural mismatch with standard Prefill–Decode (PD) disaggregation [?, ?].

PD entangles thinking with answer decode. Under PD, thinking tokens are generated on decode workers alongside answer tokens. Figure 3b quantifies this: the P90 TPOT of answer tokens crosses the 40 ms SLA at only 2.0 rps under PD, whereas the *same* answer workload in isolation could sustain 5.5 rps. Similarly, the P90 TTFAT (time-to-first-answer-token, which includes the full thinking chain) saturates at 2.9 rps under PD versus 1.9 rps for monolithic serving (Figure 3a)—an improvement, but still far below what the hardware can support.

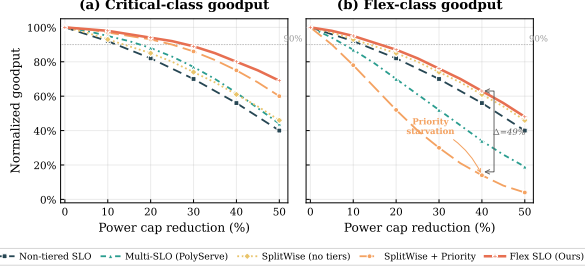


Figure 1: Goodput under demand-response power-cap reduction for five serving configurations (50/50 latency-critical/flex workload mix on a reasoning-model trace). (a) Latency-critical-class goodput: priority-based systems (SplitWise + Priority) protect critical requests at moderate reduction but still collapse past 40%. (b) Flex-class goodput: priority scheduling *starves* flex requests—only 14% goodput at 40% reduction versus 63% for Flex SLO—because it has no mechanism to exploit the relaxed latency budget. Flex SLO resolves this by letting the 2 $\times$ -relaxed tier absorb controlled slowdown, sustaining both classes simultaneously.

PTA: three-stage disaggregation. POWERSLIDER introduces a third pipeline stage: **P**refill $\rightarrow$ **T**hink $\rightarrow$ **A**nswer. Thinking tokens are generated on dedicated T-workers, completely isolating them from both the prefill pipeline and the answer decode path. This yields two compounding benefits:

1. **Prefill pipeline freedom.** Because T-workers can have thinking tokens batched with higher concurrency, the P-worker is immediately free to prefill the next request. TTFAT capacity rises to 4.4 rps (2.3 $\times$ over monolithic).
2. **Clean answer decode.** A-workers handle only answering-phase tokens, keeping TPOT flat near the SLA line up to 5.5 rps (2.8 $\times$ over PD).

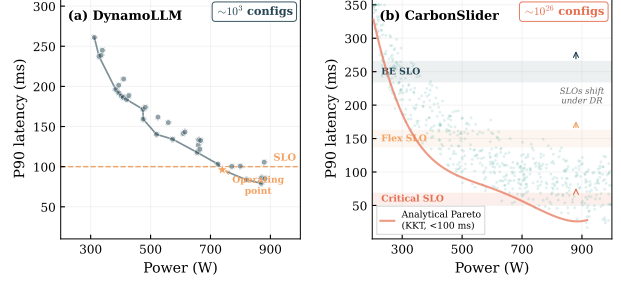


Figure 2: Configuration space explosion under demand response. (a) DynamoLLM profiles $\sim 10^3$ configurations offline in a 2D space (parallelism $\times$ frequency) and selects a single operating point that meets a fixed SLO. (b) POWERSLIDER faces a $\sim 10^{26}$ -point joint space ($k_i \times f_i \times M_i \times \text{stage} \times \text{class}$). Multi-class SLO targets (Critical, Flex, BE) shift as the power budget changes (arrows), and the dense cloud is far too large to profile exhaustively. POWERSLIDER sidesteps enumeration entirely via analytical KKT conditions that yield the Pareto front in <100 ms per epoch (red curve).

3 Problem Formulation

We formulate demand-response-aware LLM serving as a multi-objective optimization that jointly determines GPU allocation, DVFS frequency, and KV cache partitioning. The formulation directly addresses the three challenges identified in §2.5: *dynamic SLOs* (§3.2, via sliding impact bounds), the *large design space* (§3.3.2, via analytical KKT solutions that replace offline profiling), and *dual time-horizon constraints* (§3.4, via Lyapunov drift with separate power and energy queues). The section is organized progressively: §3.1 introduces the system model and decision variables, §3.2 defines the impact metric, §3.3 states the joint optimization and derives its KKT structure, and §3.4 presents the online controller.

3.1 System Model and Notation

SLO classes. We serve requests from $|\mathcal{C}|$ SLO classes, e.g., $\mathcal{C} = \{\text{LC}, \text{Flex}, \text{BE}\}$: *Latency-critical* (LC) requests require sub-second time-to-first-answer-token (TTFAT) and are rejected if the SLO cannot be met; *Flexible* (Flex) requests tolerate moderate latency inflation and may be deferred within a bounded window; *Best-effort* (BE) requests (e.g., batch summarization) have no hard latency target and can be deferred to low-carbon periods. Each class $c \in \mathcal{C}$ has arrival rate λ_c , average context length ℓ_c , and per-metric SLO targets $\{\tau_{c,m}\}$ for metrics $m \in \mathcal{M}$ (e.g., $\mathcal{M} = \{\text{TTFAT}, \text{TTLT}\}$).

Three-stage inference pipeline. Modern reasoning-capable LLMs decompose inference into three stages $\mathcal{P} =$

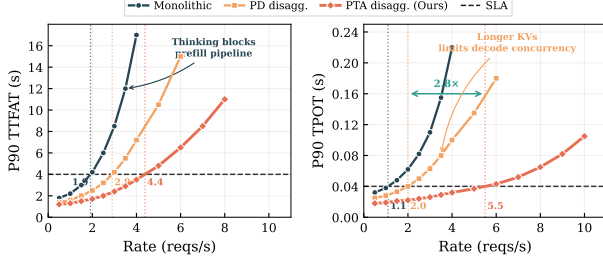


Figure 3: Serving a 13B reasoning model (input=512, thinking≈1024, output=128 tokens) on one A100-80 GB. (a) P90 time-to-first-answer-token (TTFAT): monolithic serving saturates the 4s SLA at 1.9 rps; PD disaggregation reaches 2.9 rps but thinking tokens still block the prefill pipeline. PTA pushes the crossing to 4.4 rps (2.3×) by isolating thinking on dedicated T-workers. (b) P90 TPOT: under PD, thinking tokens congest the decode workers, crossing the 40 ms SLA at only 2.0 rps. PTA’s dedicated answer workers stay flat near the SLA line, achieving 2.8× higher serving capacity (5.5 rps).

{P, T, A} (prefill, think, answer), each with distinct computational characteristics. *Prefill* (P) processes the input prompt in parallel and is predominantly compute-bound ($\gamma_P \approx 0.8$); *Think* (T) generates internal reasoning tokens (visible in some models, hidden in others) and transitions from compute-bound to memory-bandwidth-bound as the KV cache grows; *Answer* (A) produces user-visible output tokens and is typically memory-bandwidth-bound ($\gamma_A \approx 0.3$) due to large accumulated KV state. For non-reasoning models, the think stage is empty ($T_T = 0$) and the decomposition reduces to standard two-phase (prefill/decode) inference.

Stage-class index. Because each stage has a different bottleneck and may run on a separate GPU pool in a disaggregated architecture, we index decision variables by the product

$$i \in \mathcal{P} \times \mathcal{C}, \quad (3)$$

so that $i = (T, LC)$ denotes the thinking stage of latency-critical requests. This indexing is central: it lets the optimizer assign different GPU counts, frequencies, and cache budgets to each combination.

Decision variables. For each stage-class i , the system allocates three resource types:

- $k_i \in \mathcal{K}_i$: number of GPUs assigned to that pool (discrete),
- $f_i \in \mathcal{F}_i$: GPU clock frequency (discrete, 210–1410 MHz in 15 MHz steps),
- $M_i \geq 0$: KV cache memory capacity available at that stage-class (continuous),

subject to global resource constraints $\sum_i k_i \leq K$ (total GPUs) and $\sum_i M_i \leq M$ (total memory across all GPU pools). The full decision vector is $a = (\mathbf{k}, \mathbf{f}, \mathbf{M})$.

Stage-class throughput. The effective throughput (maximum sustainable request rate) of stage-class i decomposes multiplicatively:

$$s_i(k_i, f_i) = \underbrace{k_i^{\beta_i}}_{\text{parallelism}} \cdot \underbrace{[\gamma_i f_i / f_{\max} + (1 - \gamma_i)]}_{\text{frequency scaling}}, \quad (4)$$

where $\beta_i \in (0.7, 1)$ captures communication overhead in tensor/pipeline parallelism and $\gamma_i \in (0, 1)$ is the compute-bound fraction of stage i . The frequency scaling term reflects that compute-bound stages (high γ_i) benefit linearly from higher frequency, while memory-bandwidth-bound stages (low γ_i) see diminishing returns.

Cache model. Analysis of production LLM serving traces reveals that cache hit probability follows a concave power law in allocated capacity (Figure ??):

$$P_i(M_i) = \min(1, \kappa_i M_i^{\alpha_i}), \quad \alpha_i \in (0, 1), \quad (5)$$

where $\kappa_i > 0$ and α_i are workload-specific parameters fit from trace data. The concavity ($\alpha_i < 1$) is important: it means cache allocation exhibits diminishing returns, so over-provisioning one stage-class wastes capacity that could benefit another—a property that the water-filling solution (§3.3.2) exploits.

The expected per-request computation time at stage-class i is:

$$T_i(M_i) = \ell_i [a_i (1 - P_i(M_i)) + b_i], \quad (6)$$

where a_i is the per-token prefill cost (incurred on cache misses) and b_i is the per-token decode cost (incurred regardless). A higher cache hit rate P_i reduces the effective computation time by avoiding redundant prefill, and this reduction is stage-dependent: it matters most at the prefill stage (large a_i) and less at decode stages (small a_i , large b_i).

Effective capacity. Combining throughput and computation time, the *effective capacity* (maximum sustainable request rate) of stage-class i is:

$$\text{cap}_i(k_i, f_i, M_i) = \frac{s_i(k_i, f_i)}{T_i(M_i)}. \quad (7)$$

This definition links all three decision variables to a single capacity abstraction: adding GPUs or raising frequency increases the numerator; improving cache hit rate decreases the denominator. The impact model (§3.2) is defined over this capacity abstraction.

Power model. Per-GPU power consumption follows the standard cubic DVFS model for CMOS circuits: $c_i(f_i) =$

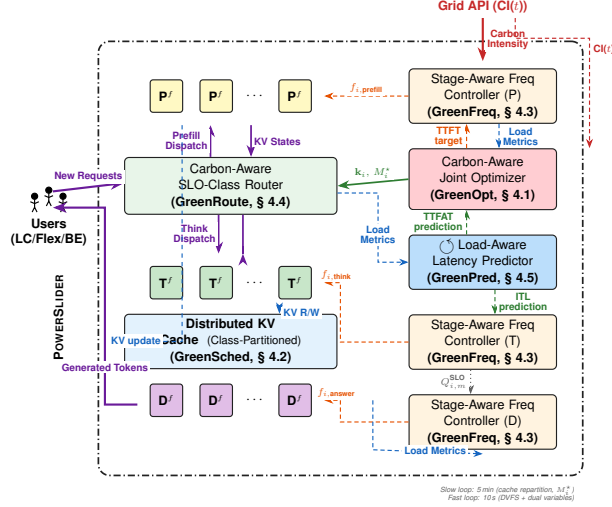


Figure 4: Design Overview of CarbonSlider YL Add More

Table 2: Formulation–design mapping.

Component	Decides	Equations	Loop
GreenOpt	$\mathbf{k}, \mathbf{M}, \mathbf{f}^{\text{init}}$	(10),(14),(15),(19)	5 min
GreenFreq	$\mathbf{f}$ refinement	(22)–(26)	10 s
GreenRoute	dispatch, admit/defer	Alg. 2	per-req
GreenSched	pool sizing, migration	Alg. 1	5 min
GreenPred	$\nabla_{f, M} I_i$, load PDF	—	10 s

where prefill, think, and answer stages may run on separate GPU pools [20], enabling independent DVFS policies and cache budgets per stage.

Each design component directly addresses one or more of the three challenges from §2.5: **GreenOpt**’s analytical KKT solver replaces offline profiling (Challenge 2) and computes dynamic SLO budgets that slide with the power constraint (Challenge 1). **GreenFreq**’s fast dual-variable updates with separate power and energy queues balance short-term power caps and long-term energy budgets (Challenge 3). **GreenRoute**’s adaptive SLO budget tightens per-class targets during DR windows (Challenge 1). **GreenPred**’s online gradient estimation enables the KKT solver to operate without pre-profiled lookup tables (Challenge 2). Table 2 maps each formulation element to its system realization.

4.1 GreenOpt: Joint Optimizer (Slow)

GreenOpt is the central decision-maker, running every 5 minutes and jointly solving for $(\mathbf{k}^*, \mathbf{M}^*, \mathbf{f}^{\text{init}})$. It ingests: (i) $\text{CI}(t)$ from the carbon signal service, (ii) per-stage-class load distributions from GreenPred, and (iii) current SLO queue backlogs $\{Q_i^{\text{SLO}}\}$.

Solver. GreenOpt solves the continuous relaxation of (19) in three steps. First, fixing $\mathbf{k}$, it uses the water-filling (14)

Algorithm 1 GreenSched: pool re-allocation (every 5 min).

Require: New targets $\{k_i^*, M_i^*\}$; current pools $\{k_i^{\text{cur}}\}$

- 1: $\Delta k_i \leftarrow k_i^* - k_i^{\text{cur}}$ for all i
- 2: *// Phase 1: Drain shrinking pools*
- 3: **for** each i with $\Delta k_i < 0$ **do**
- 4: Mark $|\Delta k_i|$ GPUs as DRAINING
- 5: Stop admitting new requests; await in-flight completion
- 6: (timeout: 30 s; preempt and recompute if exceeded)
- 7: **end for**
- 8: *// Phase 2: Migrate hot cache blocks*
- 9: **for** each drained GPU **do**
- 10: Identify top- K hot KV blocks (by access count)
- 11: Initiate background DMA to surviving GPUs (§5.4)
- 12: **end for**
- 13: *// Phase 3: Reassign GPUs to growing pools*
- 14: **for** each i with $\Delta k_i > 0$ **do**
- 15: Assign drained GPUs (prefer same NVLink island)
- 16: Push $\{M_i^*\}$ reservations to slab allocator (§??)
- 17: Update routing tables in GreenRoute
- 18: **end for**
- 19: *// Phase 4: Warm up new pool members*
- 20: **for** each newly assigned GPU **do**
- 21: Load model weights (overlapped with Phase 2 DMA)
- 22: Seed cache with high-frequency prefixes from GreenPred
- 23: **end for**

for cache and KKT (15) for frequency. Second, it enumerates feasible GPU allocations (pruned by dominance) and selects $\mathbf{k}^*$ minimizing $J(t)$. With 9 stage-classes and ~ 10 GPU-count options each, pruned enumeration completes in < 100 ms. Rounding from continuous $\mathbf{f}^*$ to 15 MHz steps introduces $< 3\%$ gap.

Carbon signal. A gRPC service hedges across Electricity Maps and WattTime; first response within 2 s wins, smoothed via EMA ($\alpha = 0.3$). A persistence-based 24-hour forecast enables deferral of BE work to anticipated low-CI windows. If signals are unavailable for > 15 min, GreenOpt degrades to power-only mode ($\text{CI} = \bar{\text{CI}}$).

Outputs. (i) Cache budgets $\{M_i^*\}$ to GreenRoute, (ii) GPU pool sizes $\{k_i^*\}$ to GreenSched, (iii) initial frequencies $\{f_i^{\text{init}}\}$ to GreenFreq.

4.2 GreenSched: Capacity Planning

GreenSched translates GreenOpt’s abstract $\{k_i^*\}$ into concrete GPU pool assignments, handling physical constraints the formulation abstracts away.

Drain-before-reassign. Abruptly stopping a GPU with in-flight decode requests would corrupt KV state and force recomputation. GreenSched first *drains* departing GPUs (no new requests; in-flight complete within 30 s timeout), then reassigns them. This adds 5–15 s to re-allocation but avoids all recomputation.

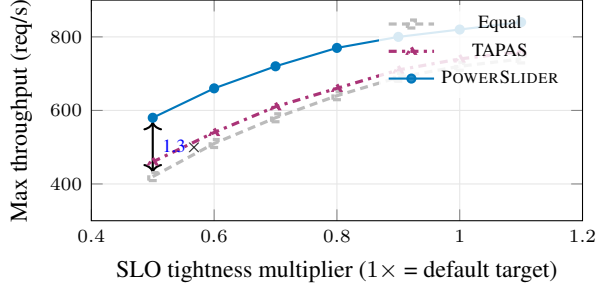


Figure 10: Maximum sustainable throughput as SLO targets tighten ($0.5\times$ = half the default target, i.e., stricter). POWERSLIDER achieves $1.3\times$ higher throughput than TAPAS under tight SLOs because cache-aware allocation reduces queueing.

The throughput gain comes from two sources. First, the water-filling cache allocation places cache where it most reduces queueing delay (high- η_i stage-classes), effectively increasing the cluster’s *usable* capacity. Second, stage-aware DVFS avoids throttling the latency-critical prefill stage, preserving TTFAT headroom that would otherwise be the binding constraint.

6.9 Controller Dynamics (RQ4)

Figure 11 traces the controller’s response to a sudden workload shift (Conv arrival rate doubles at $t = 0$). The three panels show the interplay of the two timescales:

Fast loop (seconds). Within the first 10 s dual-variable update, the SLO queue $Q_{(P,LC)}^{SLO}$ spikes (top panel), which via Eq. 25 increases $\eta_{(P,LC)}$, which via Eq. 26 immediately raises prefill frequency from 1,200 to 1,380 MHz (bottom panel). Simultaneously, the answer-stage frequency drops to 630 MHz, reclaiming power headroom under the global budget. The fast loop makes ~ 12 updates before the next slow-loop epoch and achieves 80% of the necessary frequency correction within 30 s.

Slow loop (minutes). At the next 5-minute epoch, GreenOpt re-solves the joint problem with updated load distributions. The new water-filling allocation (middle panel) shifts cache from Synth (33 $\rightarrow$ 18%) to Conv (33 $\rightarrow$ 50%), and GreenSched reassigns 4 GPUs from the Synth pool to Conv’s prefill pool. After the slow-loop commit, the SLO queue drains to baseline within ~ 1 minute.

The system reaches 95% of the new steady-state optimum within ~ 2 minutes (~ 12 fast-loop iterations + one slow-loop epoch), consistent with the $O(V^2)$ convergence bound of Theorem 1 at $V = 100$.

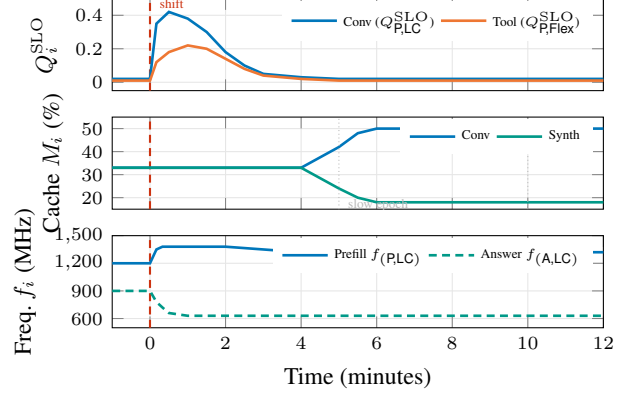


Figure 11: Controller dynamics after a workload shift (Conv arrival rate doubles at $t = 0$). **Top:** SLO virtual queues spike then drain as the controller adapts. **Middle:** Cache re-allocation commits at the next slow-loop epoch (5 min), shifting capacity from Synth to Conv. **Bottom:** Frequency adjusts within seconds via the fast loop—prefill frequency rises immediately to protect TTFAT, while answer frequency drops to reclaim power headroom. The system reaches 95% of the new optimal within ~ 2 minutes.

6.10 Sensitivity Analysis (RQ4)

We sweep key control parameters to verify robustness. Figure 12 shows the trade-off parameter V and step size η .

Trade-off parameter V . V controls the weight on the carbon-cost objective relative to constraint satisfaction. At $V = 100$, POWERSLIDER achieves $<2\%$ gap from the offline optimum with $<1\%$ constraint violation. Below $V = 50$, the gap exceeds 4% because the controller is too aggressive in maintaining constraints at the expense of carbon optimization. Above $V = 150$, diminishing returns set in.

Step size η . Larger η speeds convergence but increases overshoot (Figure 12, right). $\eta = 0.1$ provides a good balance: 2 min convergence with $<3\%$ overshoot. At $\eta = 0.25$, convergence is faster (1 min) but overshoot reaches 7%, causing transient SLO violations.

Carbon forecast error. YL *Minor, can skip* We inject Gaussian noise into the 6-hour CI forecast with varying RMSE. At 15% RMSE (typical for day-ahead forecasts), carbon savings degrade by only 8% relative to perfect foresight. At 30% RMSE, degradation is 14%—still 13 pp better than DynamoLLM. The robustness arises because POWERSLIDER’s reactive fast loop corrects for forecast errors within 10 s, and only deferral decisions (which affect 25% of savings) rely on forecasts.

- en/fast-mode, February 2026. Accessed: 2026-02-13.
- [4] James Chen. Conditional value at risk (cvar): Definition, uses, formula. *Investopedia*, 2024.
 - [5] Siyuan Chen, Zhipeng Jia, Samira Khan, Arvind Krishnamurthy, and Phillip B. Gibbons. SLOs-Serve: Optimized serving of multi-SLO LLMs. *arXiv preprint arXiv:2504.08784*, 2025.
 - [6] Jae-Won Chung, Jie You, Yile Yoo, Ramachandran Ramjee, and Mosharaf Chowdhury. Perseus: Removing energy bloat from large model training. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles*, pages 1–18, 2024.
 - [7] Walid A Hanafy, Andrew Paulos, Qianlin Tong, Huaigu Tao, Noman Bashir, Prashant Shenoy, and David Irwin. Carbonscaler: Leveraging cloud workload elasticity for optimizing carbon-efficiency. In *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, volume 8, pages 1–31. ACM, 2024.
 - [8] Chengyin Jin, Zhuohan Zhang, Qingxuan Liu, Lianmin Zheng, Siyuan Zhuang, Xin Xie, Shiyi Cao, Ion Stoica, and Joseph E Gonzalez. Ragcache: Efficient knowledge caching for retrieval-augmented generation. *ACM Transactions on Computer Systems*, 42(1-4):1–28, 2024.
 - [9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
 - [10] Shaohong Li, Rachee Singh Acker, Xu Zhang, Waltenegus Dargie, Edward W Knightly, and Henry Hoffmann. Thunderbolt: Throughput-optimized, quality-of-service-aware power capping at scale. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 1–18, 2020.
 - [11] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 38–56. ACM, August 2024.
 - [12] Varun Mehra and Raiden Hasegawa. Supporting power grids with demand response at google data centers. Google Cloud Blog, October 2023.
 - [13] Tyler Norris, Natalie M Frick, Erin Pattison, and Dalia Patino-Echeverri. Rethinking load growth: Assessing the potential for integration of large flexible loads in us power systems. Technical report, Duke University, 2025.
 - [14] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Aashaka Shah, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. *arXiv preprint arXiv:2311.18677*, 2023.
 - [15] Moinuddin K Qureshi and Yale N Patt. Utility-based cache partitioning: A low-overhead, high-performance, runtime mechanism to partition shared caches. In *39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'06)*, pages 423–432. IEEE, 2006.
 - [16] Ana Radovanović, Ross Koningstein, Ian Schneider, Bokan Chen, Alexandre Duarte, Binz Roy, Diyue Xiao, Maya Haridasan, Patrick Hung, Nick Care, et al. Carbon-aware computing for datacenters. *IEEE Transactions on Power Systems*, 38(2):1270–1280, 2022.
 - [17] Ana Radovanović, Ross Koningstein, Ian Schneider, Bokan Chen, Alexandre Duarte, Binz Roy, Diyue Xiao, Maya Haridasan, Patrick Hung, Nick Care, Saurav Talukdar, Eric Mullen, Kendal Smith, MariEllen Cottman, and Walfredo Cirne. Carbon-aware computing for datacenters. *IEEE Transactions on Power Systems*, 38(2):1270–1280, 2023.
 - [18] Abel Souza, Noman Bashir, Jorge Murillo, Walid Hanafy, Qianlin Liang, David Irwin, and Prashant Shenoy. Ecovisor: A virtual energy system for carbon-efficient applications. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 252–265, 2023.
 - [19] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. Dynamollm: Designing llm inference clusters for performance and energy efficiency. *arXiv preprint arXiv:2408.00741*, 2024.
 - [20] Jovan Stojkovic, Chaojie Zhao, and Ana Klimovic. Tapas: Thermal-and power-aware scheduling for llm inference in cloud platforms. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.

- [21] Jovan Stojkovic, Chaojie Zhao, Dheevatsa Mudigere, Aniruddha Rao, and Ana Klimovic. Dynamollm: Designing llm inference clusters for performance and energy efficiency. In *2025 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2025.
- [22] Michael Terrell. How we’re making data centers more flexible to benefit power grids. The Keyword (Google), August 2025.
- [23] Philipp Wiesner, Ilja Behnke, Dominik Scheinert, Kordian Gontarska, and Lauritz Thamsen. Let’s wait awhile: How temporal workload shifting can reduce carbon emissions in the cloud. In *Proceedings of the 22nd International Middleware Conference*, pages 260–272, 2021.
- [24] Philipp Wiesner, Ilja Behnke, Dominik Scheinert, Kordian Gontarska, and Lauritz Thamsen. Let’s wait awhile: How temporal workload shifting can reduce carbon emissions in the cloud. In *Proceedings of the 22nd International Middleware Conference*, pages 260–272, 2021.
- [25] Qiang Wu, Qingyuan Deng, Lakshmi Ganesh, Chang-Hong Hsu, Yun Jin, Sanjeev Kumar, Bin Li, Justin Meza, and Yee Jiun Song. Dynamo: Facebook’s data center-wide power management system. *ACM SIGARCH Computer Architecture News*, 44(3):469–480, 2016.
- [26] Jiali Xing, Bilge Acun, Aditya Sundarajan, David Brooks, Manoj Chakkaravarthy, Nikky Avila, Carole-Jean Wu, and Benjamin C. Lee. Carbon responder: Coordinating demand response for the datacenter fleet. *arXiv preprint arXiv:2311.08589*, 2023.
- [27] Jie You, Jae-Won Chung, and Mosharaf Chowdhury. Zeus: Understanding and optimizing GPU energy consumption of DNN training. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 119–139, Boston, MA, April 2023. USENIX Association.
- [28] Jie You, Jae-Won Chung, and Mosharaf Chowdhury. Zeus: Understanding and optimizing GPU energy consumption of DNN training. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2023)*, pages 119–139, 2023.
- [29] Jiahuan Yu, Aryan Taneja, Junfeng Lin, and Minjia Zhang. Voltanallm: Feedback-driven frequency control and state-space routing for energy-efficient llm serving, 2025.
- [30] Chao Zhang, Shinan Li, Hao Wang, and Henry Hoffmann. Flex: High-availability datacenters with zero reserved power. In *Proceedings of the 48th Annual International Symposium on Computer Architecture*, pages 319–332, 2021.
- [31] Yijia Zhang, Daniel C. Wilson, Ioannis Ch. Paschalidis, and Ayse K. Coskun. A data center demand response policy for real-world workload scenarios in HPC. In *Proceedings of the 2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 282–287. IEEE, 2021.
- [32] Yijia Zhang, Daniel C. Wilson, Ioannis Ch. Paschalidis, and Ayse K. Coskun. Hpc data center participation in demand response: An adaptive policy with QoS assurance. *IEEE Transactions on Sustainable Computing*, 7(1):157–171, 2022.
- [33] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [34] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. *arXiv preprint arXiv:2401.09670*, 2024.
- [35] Kan Zhu, Haiyang Shi, Le Xu, Jiaxin Shan, Arvind Krishnamurthy, Baris Kasikci, and Liguang Xie. PolyServe: Efficient multi-SLO serving at scale. *arXiv preprint arXiv:2507.17769*, 2025.

A Proofs

A.1 Water-Filling Optimality (Eq. 14)

From (11), KKT on M_i gives (12). Substituting (13) and solving yields (14). Complementary slackness: $\eta_i^*(\delta_i - I_i^*) = 0$. $\square$

A.2 Convergence (Theorem 1)

Lyapunov function $\mathcal{L}(t) = \frac{1}{2}[Q_B^2 + Q_M^2 + \sum_i w_i(Q_i^{\text{SLO}})^2]$. Drift-plus-penalty: $\Delta\mathcal{L} + VJ \leq D + VJ^{\text{opt}}$. Telescope: $O(1/V)$ gap, $O(V)$ backlog, $O(V^2)$ convergence. Two-timescale: singular-perturbation argument. $\square$